Text Splitters

```
<head>
  <link rel="canonical"
href="https://python.langchain.com/v0.2/docs/concepts/" />
</head>
```

Once you've loaded documents, you'll often want to transform them to better suit your application. The simplest example is you may want to split a long document into smaller chunks that can fit into your model's context window. LangChain has a number of built-in document transformers that make it easy to split, combine, filter, and otherwise manipulate documents.

When you want to deal with long pieces of text, it is necessary to split up that text into chunks. As simple as this sounds, there is a lot of potential complexity here. Ideally, you want to keep the semantically related pieces of text together. What "semantically related" means could depend on the type of text. This notebook showcases several ways to do that.

At a high level, text splitters work as following:

1. Split the text up into small, semantically meaningful chunks (often sentences).
2. Start combining these small chunks into a larger chunk until you reach a certain size (as measured by some function).
3. Once you reach that size, make that chunk its own piece of text and then start creating a new chunk of text with some overlap (to keep context between chunks).

That means there are two different axes along which you can customize your text splitter:

1. How the text is split
2. How the chunk size is measured

Types of Text Splitters

LangChain offers many different types of `text splitters`. These all live in the `langchain-text-splitters` package.

Table columns:

- **Name**: Name of the text splitter
- **Classes**: Classes that implement this text splitter
- **Splits On**: How this text splitter splits text
- **Adds Metadata**: Whether or not this text splitter adds metadata about where each chunk came from.
- **Description**: Description of the splitter, including recommendation on when to use it.

Name	Classes	Splits On	Adds Metadata	Descripti
Recursive	RecursiveCharacterTextSplitter, RecursiveJsonSplitter	A list of user defined characters		Recursivel splits text. This splitti is trying to keep relate pieces of t next to eac other. This the `recommen way` to sta splitting te
HTML	HTMLHeaderTextSplitter, HTMLSectionSplitter	HTML specific characters	✅	Splits text based on HTML-specific characters Notably, th adds in relevant informatio about whe that chunk came from

Name	Classes	Splits On	Adds Metadata	Descripti
				(based on HTML)
Markdown	MarkdownHeaderTextSplitter	Markdown specific characters	✅	Splits text based on Markdown specific characters Notably, th adds in relevant informatio about whe that chunk came from (based on Markdown
Code	many languages	Code (Python, JS) specific characters		Splits text based on characters specific to coding languages different languages are availab to choose from.
Token	many classes	Tokens		Splits text tokens. Th exist a few different ways to

Name	Classes	Splits On	Adds Metadata	Descripti
				measure tokens.
Character	CharacterTextSplitter	A user defined character		Splits text based on a user defin character. One of the simpler methods.
[Experimental] Semantic Chunker	SemanticChunker	Sentences		First splits sentences Then combines ones next each other they are semantica similar enough. Taken fron Greg Kamradt
AI21 Semantic Text Splitter	AI21SemanticTextSplitter	✅	Identifies distinct topics that form coherent pieces of text and splits along those.	

Evaluate text splitters

You can evaluate text splitters with the Chunkviz utility created by `Greg Kamradt`. `Chunkviz` is a great tool for visualizing how your text splitter is working. It will show you how your text is being split up and help in tuning up the splitting parameters.

Other Document Transforms

Text splitting is only one example of transformations that you may want to do on documents before passing them to an LLM. Head to Integrations for documentation on built-in document transformer integrations with 3rd-party tools.

Help us out by providing feedback on this documentation page:

